ONBOARDING & MICRO-LEARNING · JALASOFT

Flujo de registro del colaborador

Cómo se registra a un colaborador, cómo lo valida su Manager y qué cambia en cada pantalla y en cada tabla de Dataverse. Con el código real que lo hace posible.

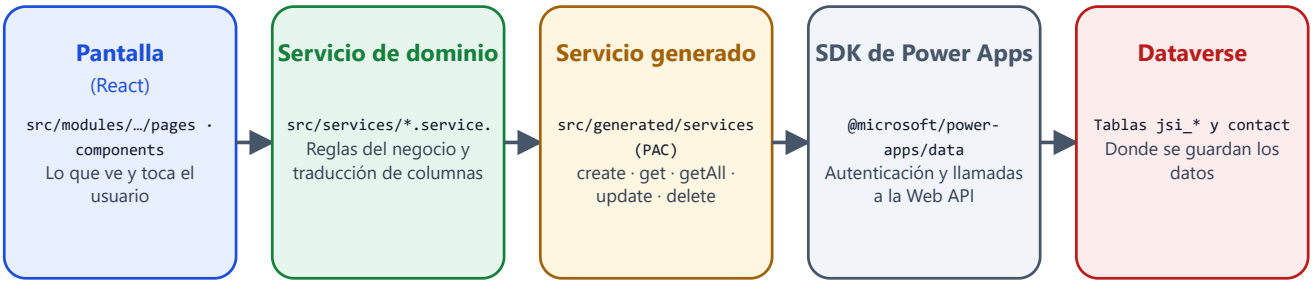
Contenido

1. Arquitectura: cómo se conecta el frontend con Dataverse
2. Las tablas que intervienen (modelo entidad-relación)
3. Paso 1 · El Administrador registra al colaborador
4. Qué cambia en cada pantalla después del registro
5. Paso 2 · El Manager valida la ruta
6. Qué cambia después de la validación
7. Cómo se calcula el estado de cada colaborador
8. Resumen y glosario

Generado el 14 de octubre de 2026 a partir del código fuente del proyecto. Tecnologías: Power Apps Code App (React + TypeScript + Vite) y Microsoft Dataverse.

1. Arquitectura: cómo se conecta el frontend con Dataverse

La aplicación es una **Power Apps Code App**: una página web hecha con React y TypeScript que corre dentro de Power Apps. No tiene un servidor propio: el "backend" es **Dataverse**, la base de datos de Power Platform, a la que se accede a través del SDK de Power Apps con la cuenta del usuario conectado.



Cada capa solo habla con la de al lado: la pantalla nunca llama a Dataverse directamente.
Los datos vuelven por el mismo camino (en sentido contrario) ya convertidos a modelos simples.

Capa	Carpeta	Responsabilidad
Pantalla	`src/modules/<rol>/pages`	Dibuja la interfaz, guarda el estado del formulario y llama a un servicio. No conoce nombres de columnas de Dataverse.
Servicio de dominio	`src/services`	Aplica las reglas del negocio (qué documentos van en la ruta, qué se crea al registrar) y traduce columnas como `_jsi_area_value` a modelos simples (`Collaborator`, `RouteStage`).
Servicio generado	`src/generated`	Lo genera el comando `pac code add-data-source`. Ofrece `create`, `get`, `getAll`, `update` y `delete` para cada tabla. No se edita a mano.
SDK de Power Apps	`@microsoft/power-apps`	Envía las peticiones a la Web API de Dataverse con la sesión del usuario.
Dataverse	Tablas `jsi_*` y `contact`	Guarda los datos y aplica los permisos.

Ejemplo: el servicio generado para la tabla contact

Así se ve un servicio que crea PAC. Recibe el registro y lo envía con `createRecordAsync`. La respuesta es un `IOperationResult`, que indica si salió bien (`success`) y trae los datos (`data`) o el error.

src/generated/services/ContactsService.ts · líneas 20–26

```ts
public static async create(record: Omit<ContactsBase, 'address1_addressid'>):
Promise<IOperationResult<Contacts>> {
  const result = await ContactsService.client.createRecordAsync<Omit<ContactsBase, 'address1_addressid'>,
Contacts>(
    ContactsService.dataSourceName,
    record
  );
  return result;
}
```

Servicio generado por PAC: no se modifica; se regenera si cambian las columnas de la tabla.

Dos ayudantes que usan todos los servicios

`unwrap` convierte el resultado en datos o en un error con un mensaje claro, y `bind` arma el valor para llenar una columna de búsqueda (un "lookup", es decir, una clave foránea).

src/services/dataverse.ts · líneas 13–22

```ts
/**
 * Devuelve los datos de una operación o lanza un error legible si falló.
 * @param action Descripción de lo que se intentaba hacer (aparece en el mensaje de error).
 */
export function unwrap<T>(result: IOperationResult<T>, action: string): T {
  if (!result.success) {
    throw new Error(`${action}: ${result.error?.message ?? 'error desconocido'}`);
  }
  return result.data;
}
```

src/services/dataverse.ts · líneas 49–57

```ts
/**
 * Valor para asignar una columna de búsqueda (lookup) al crear o actualizar un registro.
 *
 * @example
 * { 'jsi_Area@odata.bind': bind('jsi_areas', areaId) }  // → "/jsi_areas(<id>)"
 */
export function bind(entitySetName: string, id: string): string {
  return `/${entitySetName}(${id})`;
}
```

Para relacionar registros, Dataverse espera "/entitySet(id)" en una propiedad terminada en @odata.bind.

2. Las tablas que intervienen

El modelo separa tres conceptos: la **persona** (`contact`, existe una sola vez), sus **roles** (`jsi_roleassignment`, puede tener varios: Administrador, Manager, Colaborador) y cada **inducción** (`jsi_onboarding`). Así, cuando un colaborador asciende a Manager, su inducción queda como historial.

Se crean al registrar · Se crean al validar · Catálogo (ya existen)

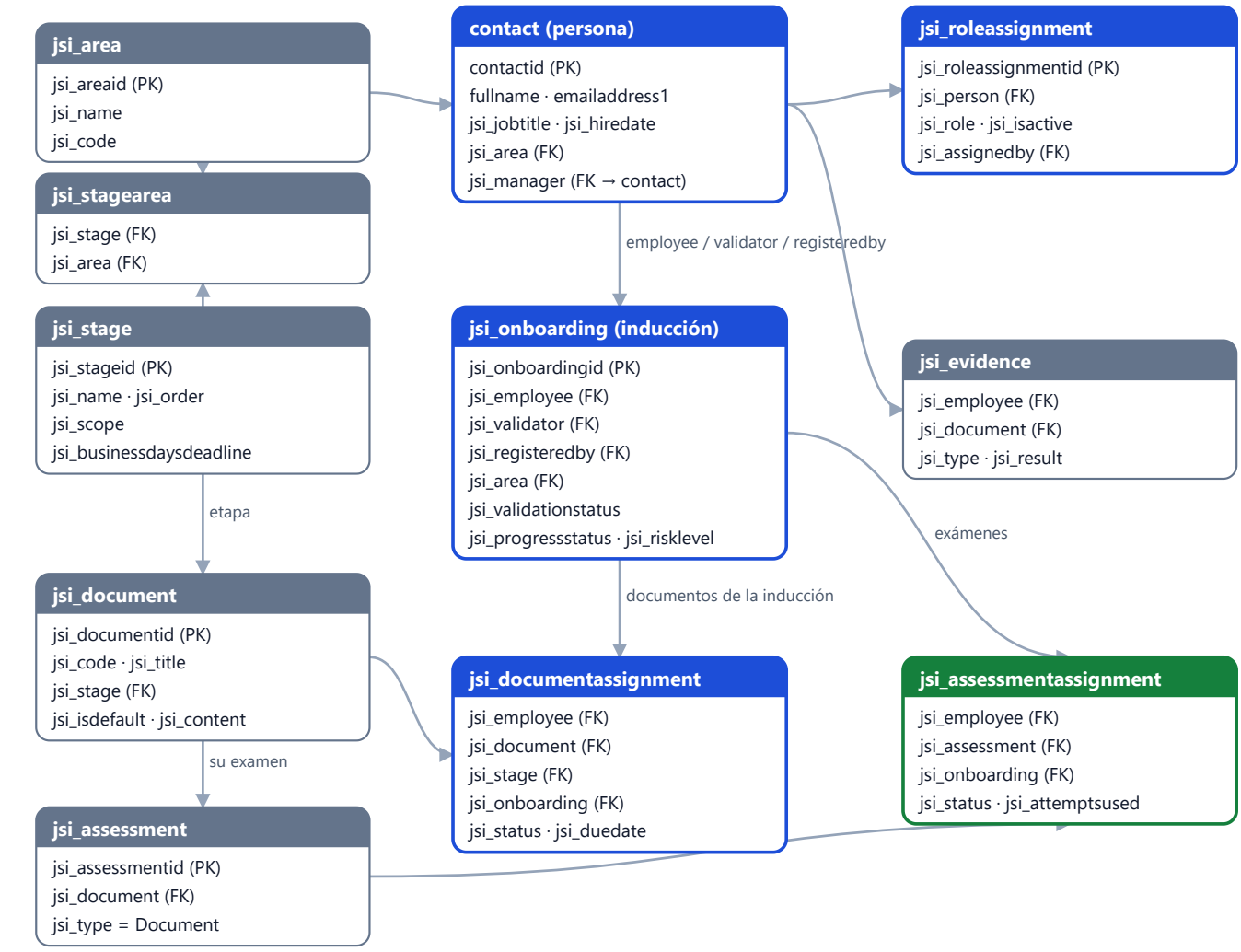
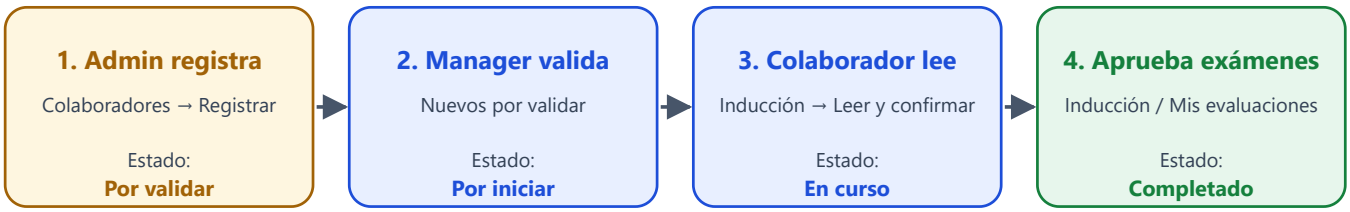


Tabla	Para qué sirve en este flujo
`contact`	La persona: nombre, correo (único), cargo, fecha de ingreso, área y su Manager directo (`jsi_manager`).
`jsi_roleassignment`	Su rol activo. Al registrar se crea el rol *Collaborator*; con él aparece en el inicio de sesión como colaborador.
`jsi_onboarding`	La inducción: quién la valida (`jsi_validator`), quién la registró (`jsi_registeredby`), el área cuya ruta se usó y su estado (`jsi_validationstatus`, `jsi_progressstatus`, `jsi_risklevel`).
`jsi_documentassignment`	Un documento de su ruta: estado de lectura (*Pending* / *Read*) y vencimiento.

`jsi_assessmentassignment`	El examen de cada documento: *Locked* → *Pending* → *Passed* / *Failed*, con intentos usados.
`jsi_stage`, `jsi_stagearea`, `jsi_document`, `jsi_assessment`	Catálogo: las etapas (1 y 2 generales, una Etapa 3 por área), los documentos con su contenido y la evaluación de cada documento.

> El modelo completo (23 tablas y 41 relaciones) está en `docs/modelo-entidad-relacion.md` y su diagrama interactivo en `docs/modelo-er-grafico.html`.

3. Paso 1 · El Administrador registra al colaborador



Dónde: el Administrador entra por el selector de inicio, va a **Colaboradores** y pulsa **Registrar colaborador**. Completa nombres, apellidos, correo, cargo, área, Manager responsable y fecha de ingreso.

3.1 La ruta propuesta se calcula en el navegador

Al elegir el área, el formulario arma la ruta con el catálogo ya leído de Dataverse. Primero `buildAreaRoute` junta todas las etapas que aplican al área: las generales (1 y 2) y la Etapa 3 de esa área.

src/services/onboardingRoute.service.ts · líneas 144–160

```ts
export function buildAreaRoute(catalog: OnboardingCatalog, areaId: string): RouteStage[] {
  const appliesToArea = (areas: Set<string> | undefined) => areas?.has(areaId) ?? false;

  return sortStages(catalog.stages)
    .filter((stage) => stage.isGeneral || appliesToArea(catalog.stageAreas.get(stage.id)))
    .map((stage) => ({
      id: stage.id,
      name: stage.name,
      order: stage.order,
      isGeneral: stage.isGeneral,
      businessDays: stage.businessDays,
      documents: documentsOfStage(catalog, stage.id).filter(
        (doc) => !catalog.documentAreas.has(doc.id) || appliesToArea(catalog.documentAreas.get(doc.id)),
      ),
    }))
    .filter((stage) => stage.documents.length > 0);
}
```

Después `proposedDocumentIds` decide qué documentos se asignan: todos los de las etapas generales y, de la Etapa 3, solo los marcados como "por defecto" (`jsi_isdefault`). Los demás quedan como opcionales para el Manager.

src/services/onboardingRoute.service.ts · líneas 176–181

```ts
/** Documentos que el Admin propone: todos los de las etapas generales y los "por defecto" de la Etapa 3. */
export function proposedDocumentIds(areaRoute: RouteStage[]): Set<string> {
  return new Set(
    areaRoute.flatMap((stage) => stage.documents.filter((d) => stage.isGeneral || d.isDefault).map((d) => d.id)),
  );
}
```

src/modules/admin/pages/collaborators/components/RegisterCollaboratorModal.tsx · líneas 65–70

```
65 // Todas las etapas del área y, dentro de ellas, los documentos que el Admin propone.
66 const areaRoute = useMemo(
67   () => (catalog.data && form.areaId ? buildAreaRoute(catalog.data, form.areaId) : []),
68   [catalog.data, form.areaId],
69 );
70 const proposedIds = useMemo(() => proposedDocumentIds(areaRoute), [areaRoute]);
```

En la pantalla: useMemo recalcula la ruta solo cuando cambia el catálogo o el área elegida.

3.2 Al pulsar “Registrar”

La pantalla valida el formulario y llama al servicio `registerCollaborator` con los datos, la ruta propuesta y el **id del Administrador** que está registrando (sale de la sesión).

src/modules/admin/pages/collaborators/components/RegisterCollaboratorModal.tsx · líneas 83–97

```
83 const handleSubmit = async (event: FormEvent) => {
84   event.preventDefault();
85   setShowErrors(true);
86   if (Object.keys(validate(form)).length > 0 || !catalog.data) return;
87
88   setSubmitting(true);
89   setSubmitError(null);
90   try {
91     await registerCollaborator(form, filterRoute(areaRoute, proposedIds), admin.id);
92     onRegistered(`${form.firstName.trim()} ${form.lastName.trim()}`);
93   } catch (error) {
94     setSubmitError(error instanceof Error ? error.message : String(error));
95     setSubmitting(false);
96   }
97 };
```

3.3 El servicio crea los registros en Dataverse

Primero comprueba que el correo no exista (el correo identifica a la persona). Luego crea la **persona** en `contact`, con su área y su Manager como lookups.

src/services/collaborators.service.ts · líneas 167–194

```ts
export async function registerCollaborator(
  input: NewCollaboratorInput,
  route: RouteStage[],
  registeredById: string,
): Promise<string> {
  const email = input.email.trim().toLowerCase();
  if (await isEmailRegistered(email)) {
    throw new Error(`Ya existe una persona registrada con el correo ${email}.`);
  }

  // Acciones para deshacer lo creado, en orden inverso, si algo falla.
  const undo: (() => Promise<unknown>)[] = [];

  try {
    const person = unwrap(
      await ContactsService.create({
        firstname: input.firstName.trim(),
        lastname: input.lastName.trim(),
        emailaddress1: email,
        jsi_jobtitle: input.jobTitle.trim(),
        jsi_hiredate: input.hireDate,
        'jsi_Area@odata.bind': bind('jsi_areas', input.areaId),
        'jsi_Manager@odata.bind': bind('contacts', input.managerId),
      } as NewContactRecord),
      'Registrar persona',
    );
    const personId = person.contactid;
    undo.push(() => ContactsService.delete(personId));
```

Después crea su **rol** de Colaborador y su **inducción**, pendiente de validación, con el Manager como validador y el Administrador como quien la registró.

src/services/collaborators.service.ts · líneas 196–223

```ts
const role = unwrap(
  await Jsi_roleassignmentsService.create({
    'jsi_Person@odata.bind': bind('contacts', personId),
    jsi_role: ROLE.Collaborator,
    jsi_isactive: true,
    jsi_startdate: input.hireDate,
    jsi_jobtitle: input.jobTitle.trim(),
  } as NewRoleRecord),
  'Asignar rol de colaborador',
);
undo.push(() => Jsi_roleassignmentsService.delete(role.jsi_roleassignmentid));

const onboarding = unwrap(
  await Jsi_onboardingsService.create({
    'jsi_Employee@odata.bind': bind('contacts', personId),
    'jsi_Validator@odata.bind': bind('contacts', input.managerId),
    'jsi_RegisteredBy@odata.bind': bind('contacts', registeredById),
    'jsi_Area@odata.bind': bind('jsi_areas', input.areaId),
    jsi_type: ONBOARDING_TYPE.Initial,
    jsi_validationstatus: VALIDATION_STATUS.PendingValidation,
    jsi_progressstatus: PROGRESS_STATUS.NotStarted,
    jsi_risklevel: RISK_LEVEL.Low,
    jsi_startdate: input.hireDate,
  } as NewOnboardingRecord),
  'Crear inducción',
);
const onboardingId = onboarding.jsi_onboardingid;
undo.push(() => Jsi_onboardingsService.delete(onboardingId));
```

Por último asigna los **documentos** de la ruta propuesta. Si algo falla, recorre la lista `undo` en orden inverso y borra lo creado, para no dejar datos a medias.

src/services/collaborators.service.ts · líneas 225–258

```
    const assignments = route.flatMap((stage) =>
      stage.documents.map(
        (document) =>
          ({
            'jsi_Employee@odata.bind': bind('contacts', personId),
            'jsi_Document@odata.bind': bind('jsi_documents', document.id),
            'jsi_Stage@odata.bind': bind('jsi_stages', stage.id),
            'jsi_Onboarding@odata.bind': bind('jsi_onboardings', onboardingId),
            jsi_status: ASSIGNMENT_STATUS.Pending,
            jsi_source: 'Propuesto por el Administrador',
          }) as NewAssignmentRecord,
      ),
    );

    const results = await Promise.allSettled(
      assignments.map(async (record) => unwrap(await Jsi_documentassignmentsService.create(record),
  'Asignar documento')),
    );
    for (const result of results) {
      if (result.status === 'fulfilled') {
        const id = result.value.jsi_documentassignmentid;
        undo.push(() => Jsi_documentassignmentsService.delete(id));
      }
    }
    const failed = results.find((result) => result.status === 'rejected');
    if (failed) throw failed.reason;

    return personId;
  } catch (error) {
    for (const action of undo.reverse()) {
      await action().catch(() => undefined);
    }
    throw error instanceof Error ? error : new Error(String(error));
  }
}
```

Los exámenes todavía no se crean: se crean cuando el Manager confirma la ruta.

3.4 Resultado en la base de datos

Tabla	Registros	Valores clave
`contact`	1	firstname, lastname, emailaddress1 (en minúsculas), jsi_jobtitle, jsi_hiredate, jsi_area, jsi_manager
`jsi_roleassignment`	1	jsi_role = *Collaborator*, jsi_isactive = Sí, jsi_startdate = fecha de ingreso
`jsi_onboarding`	1	jsi_type = *Initial*, jsi_validationstatus = *PendingValidation*, jsi_progressstatus = *NotStarted*, jsi_risklevel = *Low*, jsi_validator, jsi_registeredby
`jsi_documentassignment`	Uno por documento propuesto (ej. 14 en Engineering)	jsi_status = *Pending*, jsi_source = "Propuesto por el Administrador", sin vencimiento todavía

4. Qué cambia en cada pantalla después del registro

Ninguna pantalla guarda copias de los datos: cada una vuelve a leer Dataverse al abrirse. Por eso el nuevo registro aparece en todas, cada una con lo que le corresponde a su rol.

ADMINISTRADOR
Colaboradores

Aparece en la tabla con estado Por validar, documentos 0/14 y su Manager. Sube el contador "pendientes de validación".

ADMINISTRADOR
Panel general

Entra en "Colaboradores que requieren atención" con el motivo *Pendiente de validación de <Manager>*, y en la barra "Estado de las inducciones".

ADMINISTRADOR
Managers y ascensos

Aparece como candidato con el motivo *Su ruta aún no fue validada*: todavía no se lo puede ascender. Suma 1 en "Por validar" del equipo de su Manager.

MANAGER
Panel del equipo

Muestra el aviso *Se le asignó un nuevo colaborador para validar* con el botón **Revisar y validar**.

MANAGER
Nuevos por validar

Aparece la tarjeta del colaborador con su área, fecha de ingreso, correo y **Registrado por** (el Administrador), y su ruta por etapa.

COLABORADOR
Inicio de sesión y Mi progreso

Ya puede elegirse en el inicio como Colaborador. Al entrar ve *Su ruta de inducción todavía no está activa* y la documentación propuesta en gris.

4.1 Cómo sabe cada pantalla qué mostrar

La pantalla del Administrador calcula el estado de cada colaborador en `loadCollaboratorsOverview`: si su inducción está pendiente de validación, el estado es *pendingValidation* ("Por validar"); si no, se calcula con su ruta real.

src/services/collaborators.service.ts · líneas 95–99

```
95  const entries = onboarding ? (routes.get(onboarding.jsi_onboardingid) ?? []) : [];
96  const progress = progressOf(entries);
97  const isPending = onboarding?.jsi_validationstatus === VALIDATION_STATUS.PendingValidation;
98  const status = isPending ? 'pendingValidation' : progress.status;
99  const managerName = (personManagerId && people.get(personManagerId)?.fullname) || '—';
```

El colaborador, en cambio, recibe una respuesta distinta según el estado de su inducción. Si está pendiente, solo se le muestran los documentos propuestos:

src/services/myInduction.service.ts · líneas 94–100

```ts
if (onboarding.jsi_validationstatus === VALIDATION_STATUS.PendingValidation) {
  const proposed = entries.flatMap((entry) => {
    const item = documents.get(entry.documentId);
    return item ? [item.document] : [];
  });
  return { ...info, state: 'pendingValidation', proposed };
}
```

Además, cada rol solo puede abrir sus propias pantallas: el layout comprueba la sesión antes de dibujar.

src/layouts/AppLayout/AppLayout.tsx · líneas 21–24

```tsx
// Solo se entra a las pantallas de un rol con una sesión de ese mismo rol.
if (!session || session.role !== module.role) {
  return <Navigate to="/" replace />;
}
```

5. Paso 2 · El Manager valida la ruta

Dónde: el Manager entra con su usuario, va a **Nuevos por validar** y ve una tarjeta por cada colaborador que le toca validar. La consulta filtra por `jsi_validator` = el Manager con sesión y por estado *PendingValidation*:

src/services/validation.service.ts · líneas 39–48

```
39 export async function loadPendingValidations(managerId: string): Promise<PendingOnboarding[]> {
40   const [onboardings, people, areas, catalog] = await Promise.all([
41     fetchAll(
42       (o) => Jsi_onboardingsService.getAll(o),
43       {
44         select: ['jsi_onboardingid', '_jsi_employee_value', '_jsi_area_value', '_jsi_registeredby_value',
   'jsi_startdate'],
45         filter:
46           `${ACTIVE_RECORDS} and _jsi_validator_value eq ${managerId}` +
47           ` and jsi_validationstatus eq ${VALIDATION_STATUS.PendingValidation}`,
48         orderBy: ['createdon asc'],
```

5.1 Qué puede cambiar el Manager

Las Etapas 1 y 2 aparecen tildadas y bloqueadas (“Fija”). Solo la Etapa 3 de su área es editable: puede quitar los documentos que envió el Administrador o sumar los demás del área.

src/modules/manager/pages/pending-validation/components/PendingValidationCard.tsx · líneas 94–123

```tsx
{areaRoute.map((stage) => {
  const editable = !stage.isGeneral;
  const marked = stage.documents.filter((d) => allSelected.has(d.id)).length;
  return (
    <StageGroup
      key={stage.id}
      name={stage.name}
      summary={
        <span className="pending-card__stage-summary">
          {editable ? `${marked} de ${stage.documents.length} seleccionados` : `${stage.documents.length} documentos obligatorios`}
          <Badge tone={editable ? 'info' : 'neutral'}>{editable ? 'Editable por el Manager' : 'Fija'}</Badge>
        </span>
      }
    >
      {stage.documents.map((document) => {
        const inputId = `${pending.onboardingId}-${document.id}`;
        const isSelected = allSelected.has(document.id);
        return (
          <li key={document.id}>
            <DocumentItem
              document={document}
              htmlFor={editable ? inputId : undefined}
              muted={!isSelected}
              leading={
                <input
                  id={inputId}
                  type="checkbox"
                  className="doc-item__check"
                  checked={isSelected}
                  disabled={!editable || saving}
```

La casilla se desactiva cuando la etapa no es editable (etapas generales).

Antes de confirmar se controla que quede al menos un documento de la Etapa 3:

src/services/validation.service.ts · líneas 106–112

```ts
export function validateSelection(areaRoute: RouteStage[], selected: Set<string>): string | null {
  const areaStages = areaRoute.filter((stage) => !stage.isGeneral);
  const hasAreaDocuments = areaStages.some((stage) => stage.documents.length > 0);
  const selectedInArea = areaStages.some((stage) => stage.documents.some((d) => selected.has(d.id)));
  if (hasAreaDocuments && !selectedInArea) return 'Seleccione al menos un documento de la Etapa 3 de su área.';
  return null;
}
```

5.2 Al pulsar “Confirmar y activar ruta”

src/services/validation.service.ts · líneas 125–189

```ts
export async function confirmOnboarding(pending: PendingOnboarding, selectedEtapa3: Set<string>): Promise<void> {
  // Las etapas generales siempre van completas; de la Etapa 3, lo que eligió el Manager.
  const selected = new Set(selectedEtapa3);
  pending.areaRoute.filter((stage) => stage.isGeneral).forEach((stage) => stage.documents.forEach((d) => selected.add(d.id)));

  const problem = validateSelection(pending.areaRoute, selected);
  if (problem) throw new Error(problem);

  // Los plazos cuentan desde la fecha de ingreso, o desde hoy si la validación llega después.
  const today = todayIso();
  const start = pending.startDate?.slice(0, 10);
  const startDate = start && start > today ? start : today;

  const route = filterRoute(pending.areaRoute, selected);
  const dueDates = computeStageDueDates(route, startDate);
  const { collaborator, onboardingId } = pending;

  // 1. Quitar
  const toRemove = [...pending.assignedDocuments].filter(([documentId]) => !selected.has(documentId));
  await Promise.all(toRemove.map(([, assignmentId]) => Jsi_documentassignmentsService.delete(assignmentId)));

  // 2, 3 y 4. Sumar, poner vencimientos y crear exámenes
  await Promise.all(
    route.flatMap((stage) =>
      stage.documents.map(async (document) => {
        const dueDate = dueDates.get(stage.id) ?? undefined;
        const assignmentId = pending.assignedDocuments.get(document.id);

        if (assignmentId) {
          const changes: AssignmentChanges = { jsi_duedate: dueDate };
          unwrap(await Jsi_documentassignmentsService.update(assignmentId, changes), 'Actualizar vencimiento');
        } else {
          const record = {
            'jsi_Employee@odata.bind': bind('contacts', collaborator.id),
            'jsi_Document@odata.bind': bind('jsi_documents', document.id),
            'jsi_Stage@odata.bind': bind('jsi_stages', stage.id),
            'jsi_Onboarding@odata.bind': bind('jsi_onboardings', onboardingId),
            jsi_status: ASSIGNMENT_STATUS.Pending,
            jsi_duedate: dueDate,
            jsi_source: 'Agregado por el Manager',
          } as NewAssignmentRecord;
          unwrap(await Jsi_documentassignmentsService.create(record), 'Asignar documento');
        }

        if (document.assessmentId) {
          const exam = {
            'jsi_Employee@odata.bind': bind('contacts', collaborator.id),
            'jsi_Assessment@odata.bind': bind('jsi_assessments', document.assessmentId),
            'jsi_Onboarding@odata.bind': bind('jsi_onboardings', onboardingId),
            jsi_status: EXAM_STATUS.Locked,
            jsi_attemptsused: 0,
          } as NewExamRecord;
          unwrap(await Jsi_assessmentassignmentsService.create(exam), 'Crear examen');
        }
```

```
      }),
    ),
  );

  // 5. Activar la ruta
  const changes: OnboardingChanges = {
    jsi_validationstatus: VALIDATION_STATUS.Validated,
    jsi_validatedon: new Date().toISOString(),
  };
  unwrap(await Jsi_onboardingsService.update(onboardingId, changes), 'Validar inducción');
}
```

Los vencimientos se acumulan por etapa: si la Etapa 1 tiene 5 días hábiles y la Etapa 2 otros 5, la primera vence a los 5 y la segunda a los 10 días hábiles desde el inicio.

src/services/onboardingRoute.service.ts · líneas 198–210

```
export function computeStageDueDates(stages: RouteStage[], startDate: string): Map<string, string | null>
{
  const dueDates = new Map<string, string | null>();
  let accumulated = 0;
  for (const stage of [...stages].sort((a, b) => a.order - b.order)) {
    if (stage.businessDays === null) {
      dueDates.set(stage.id, null);
      continue;
    }
    accumulated += stage.businessDays;
    dueDates.set(stage.id, addBusinessDays(startDate, accumulated));
  }
  return dueDates;
}
```

5.3 Resultado en la base de datos

Tabla	Cambio
`jsi_documentassignment`	Se borran los documentos destildados, se crean los sumados (jsi_source = "Agregado por el Manager") y todos reciben su **jsi_duedate**.
`jsi_assessmentassignment`	Se crea **un examen por documento** con jsi_status = *Locked* y 0 intentos usados. Se desbloquea cuando el colaborador confirma la lectura.
`jsi_onboarding`	jsi_validationstatus = *Validated* y jsi_validatedon = fecha y hora de la validación. Se actualiza al final, para que la ruta solo se active si todo lo anterior salió bien.

6. Qué cambia después de la validación

Pantalla	Antes	Después
Admin · Colaboradores	Por validar	Por iniciar, con 0/N documentos y vencimientos
Manager · Nuevos por validar	Tarjeta del colaborador	Desaparece (ya no está pendiente). Aviso de éxito.
Manager · Panel / Colaboradores	Aviso "nuevo colaborador"	Pasa a la tabla de avance con el botón **Recordar**.

Colaborador · Mi progreso	“Su ruta todavía no está activa”	Fase 2 en curso, anillo de avance y “Continuar inducción”.
Colaborador · Inducción	Documentos propuestos en gris	Lista “Por completar”: **Leer y confirmar** → **Rendir examen** → aprobado.
Colaborador · Mis evaluaciones	—	Un examen por documento, `Bloqueada` hasta confirmar la lectura.

7. Cómo se calcula el estado de cada colaborador

El estado no se escribe a mano: se calcula siempre con la misma función pura a partir de su ruta real. La usan el Administrador, el Manager y el Colaborador, por eso los tres ven los mismos números.

src/shared/domain/onboardingProgress.ts · líneas 52–79

```ts
export function computeProgress(items: RouteItemState[], today: string, maxAttempts: number): ProgressSummary {
  const steps = items.map((item) => documentStep(item, maxAttempts));
  const total = items.length;
  const approved = steps.filter((step) => step === 'approved').length;
  const read = items.filter((item) => item.isRead).length;
  const examsToTake = steps.filter((step) => step === 'exam').length;

  const scores = items.flatMap((item) => (item.exam?.score != null ? [item.exam.score] : []));
  const averageScore = scores.length ? Math.round(scores.reduce((sum, value) => sum + value, 0) / scores.length) : null;

  const hasOverdue = items.some((item) => isOverdue(item, today));
  const hasExhausted = steps.includes('exhausted');
  const hasFailed = items.some((item) => item.exam?.status === 'failed');

  let status: ProgressSummary['status'];
  if (total > 0 && approved === total) status = 'completed';
  else if (hasOverdue) status = 'overdue';
  else if (read === 0 && scores.length === 0) status = 'notStarted';
  else status = 'inProgress';

  let risk: RiskLevel = 'low';
  if (status !== 'completed') {
    if (hasOverdue || hasExhausted) risk = 'high';
    else if (hasFailed) risk = 'medium';
  }

  return { total, read, approved, examsToTake, averageScore, status, risk };
}
```

Estado	Cuándo
Por validar	La inducción está en *PendingValidation* (todavía no la confirmó el Manager).
Por iniciar	Validada, pero sin lecturas ni exámenes.
En curso	Tiene al menos una lectura o examen y le faltan documentos por aprobar.
Atrasado	Tiene una lectura o un examen con la fecha de vencimiento pasada.
Completado	Todos sus documentos están **leídos y con el examen aprobado** (mínimo 80%). Recién entonces el Administrador puede ascenderlo a Manager.

8. Resumen y glosario

1. **Admin registra** → se crean persona, rol, inducción y documentos propuestos. Estado “Por validar”.
2. **Manager valida** → ajusta la Etapa 3, se fijan vencimientos y se crean los exámenes. Estado “Por iniciar”.

3. **Colaborador** lee, confirma y aprueba cada examen → “En curso” → “Completado”.

Término	Significado
Lookup (FK)	Columna que apunta a otro registro (ej. jsi_area de un contacto). Al leerla llega como `_jsi_area_value`; al escribirla se usa `"jsi_Area@odata.bind"`.
Choice	Columna de opciones con valores numéricos (ej. 100000001 = PendingValidation). Los valores están en `src/shared/constants/choices.ts`.
PAC	Power Platform CLI. `pac code add-data-source` genera los modelos y servicios de cada tabla.
Servicio de dominio	Archivo de `src/services` con las reglas del negocio; las pantallas solo hablan con estos servicios.
Función pura	Función que solo calcula a partir de lo que recibe (sin leer ni guardar datos), como `computeProgress`. Es fácil de probar y siempre da el mismo resultado.

> Las notificaciones por correo (bienvenida, recordatorios) todavía no se envían: la app guarda los avisos en `jsi_notification` y el envío se hará más adelante con Power Automate.